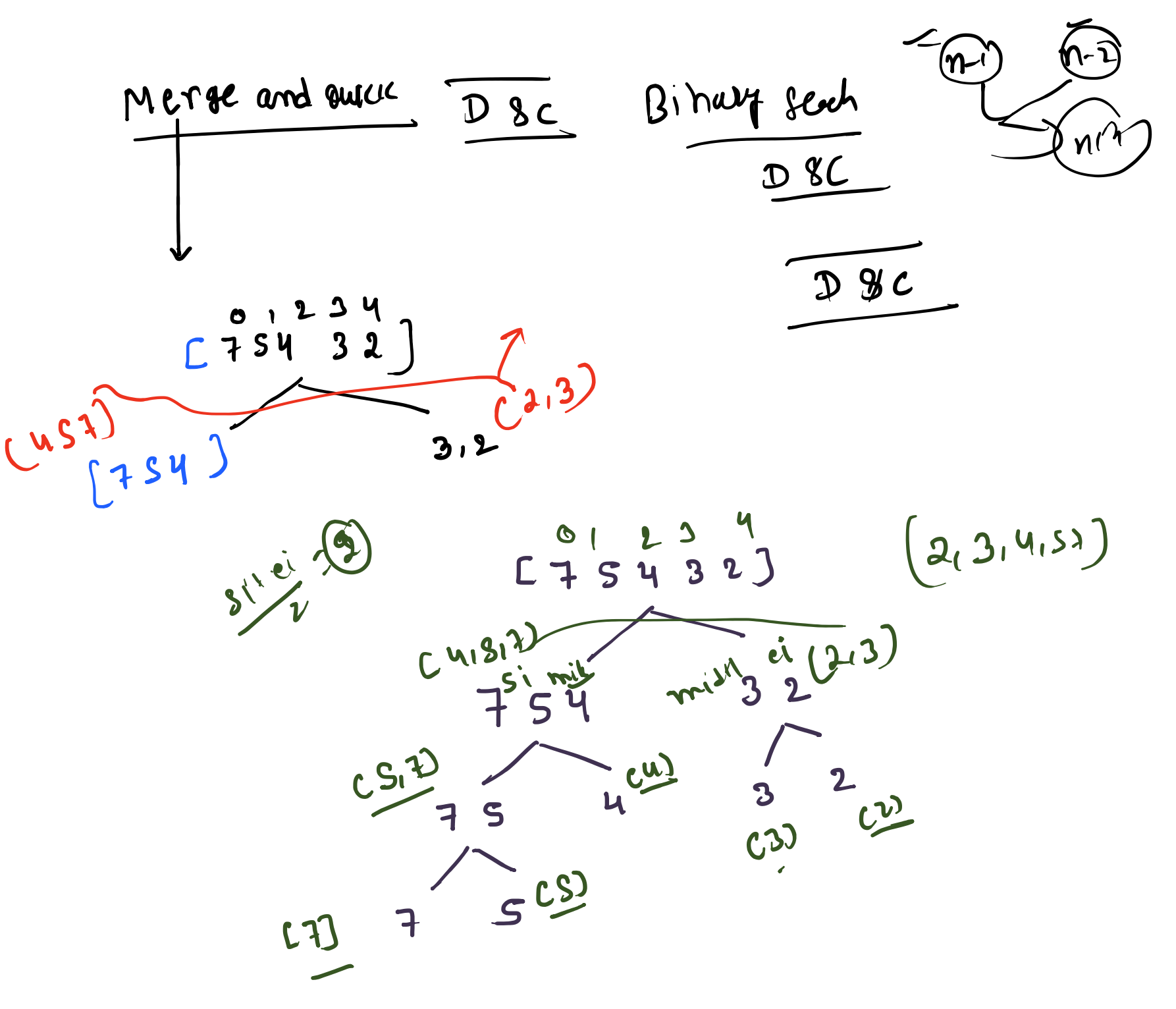
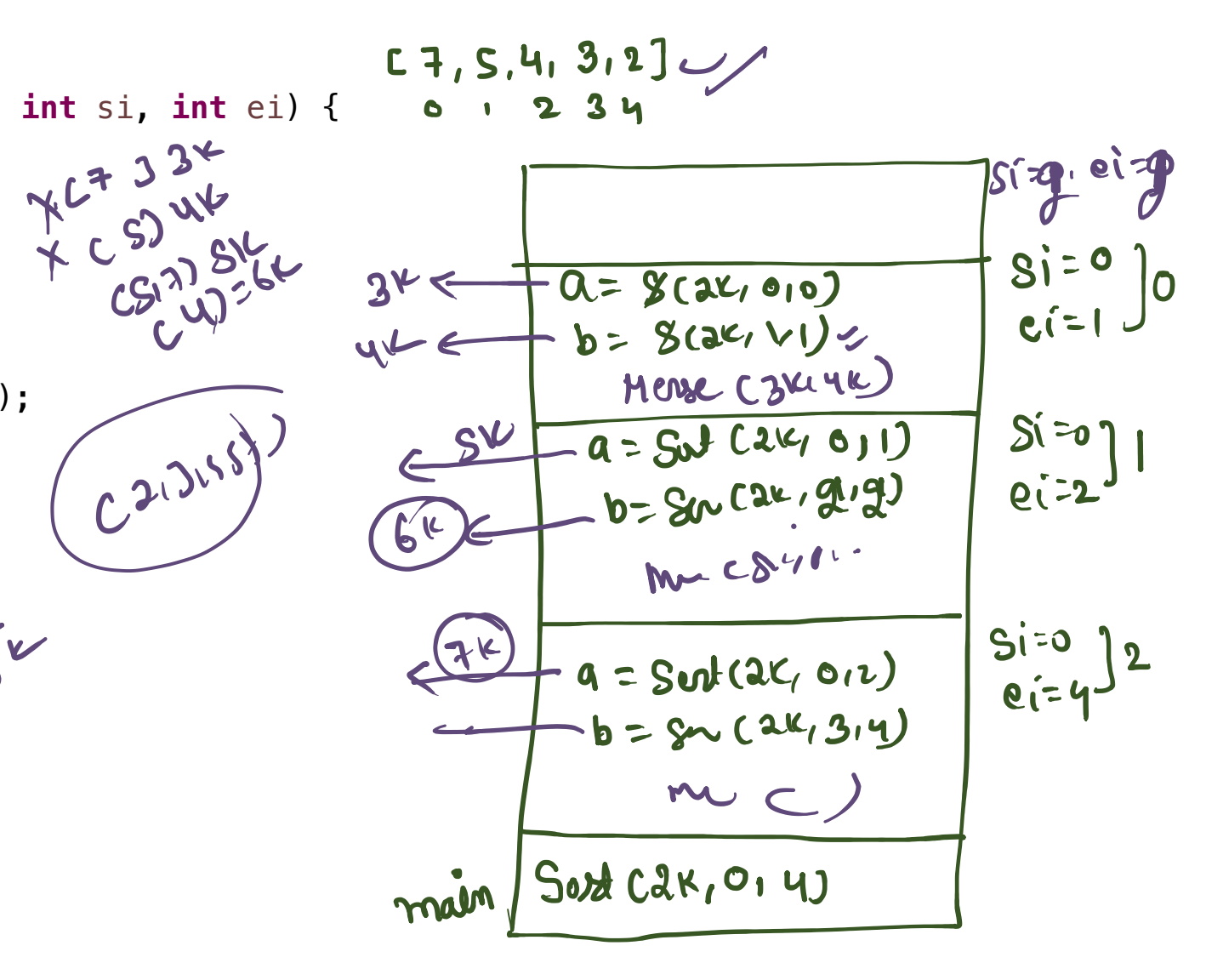
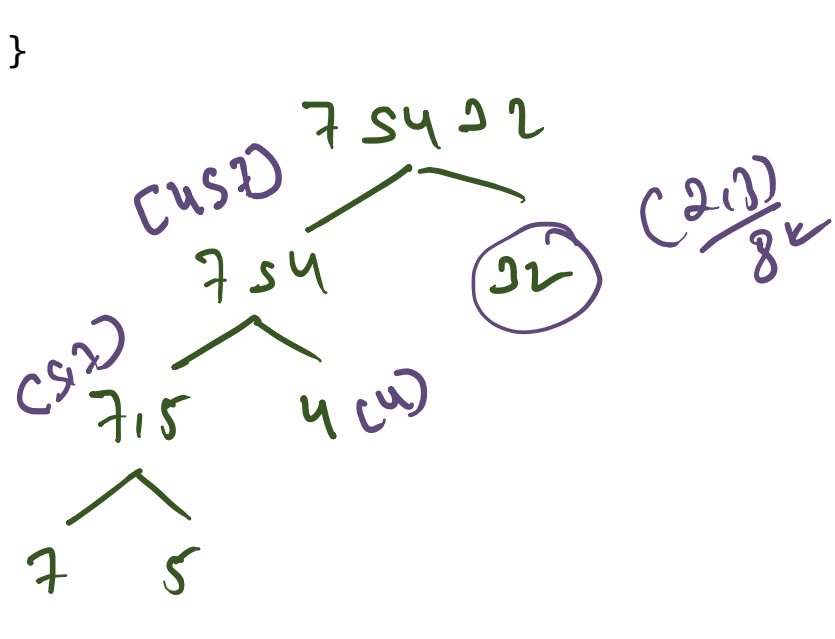
Lec-23
22/08/26 12:56 PM

Merge and Quick D&C Binary Search D&C D&C

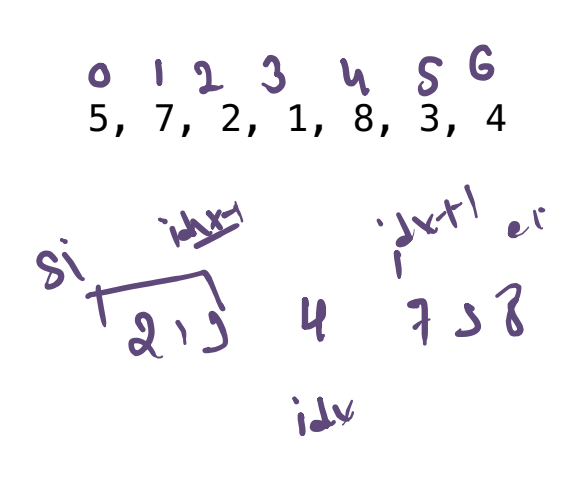

n-1
n-2
n/2
0 1 2 3 4
[7 5 4 3 2]
(4,5,7)
[7 5 4]
(2,3)
3,2
si+ei/2
(2,3,4,5,7)
si mid
mid+1 ei
7 5 4
3 2
(5,7)
7 5
(4)
(2)
[7]
5 (5)

```
public static int[] Sort(int[] arr, int si, int ei) {
    if (si == ei) {
        int[] bs = new int[1];
        bs[0]=arr[si];
        return bs;
    }
    int mid = (si + ei) / 2;
    int[] a = Sort(arr, si, mid);
    int[] b = Sort(arr, mid + 1, ei);
    return Merge_Two_Array(a, b);
}
```

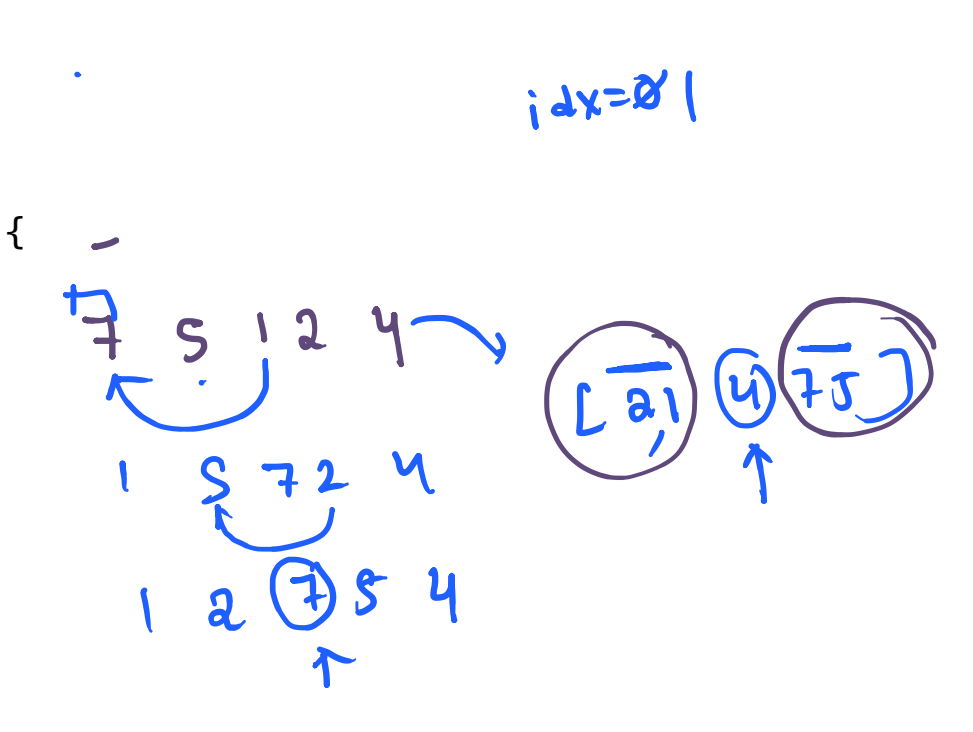

[7,5,4,3,2]
0 1 2 3 4
a = S(2K, 0,0)
b = S(2K, 1,1)
Merge (3K, 4K)
a = Sort(2K, 0,1)
b = Sort(2K, 2,2)
a = Sort(2K, 0,2)
b = Sort(2K, 3,4)
main Sort(2K, 0, 4)
si=0 ei=4
si=0 ei=1
si=0 ei=2
si=0 ei=4
3K 4K 5K 6K 7K
(2,3,4,5,7)


(4,5,7) 7 5 4 3 2
7 5 4
(2,3) 8K
(5,7)
7,5 4 (4)
7 5

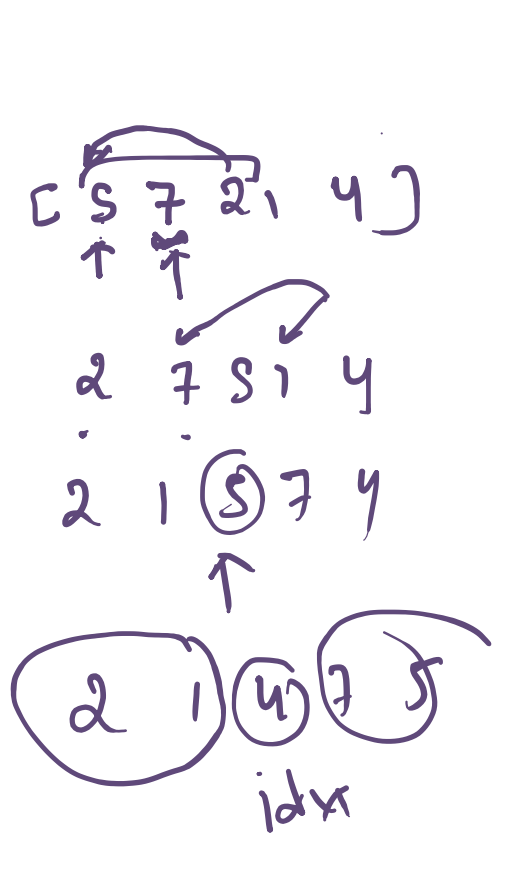
0 1 2 3 4 5 6
5, 7, 2, 1, 8, 3, 4

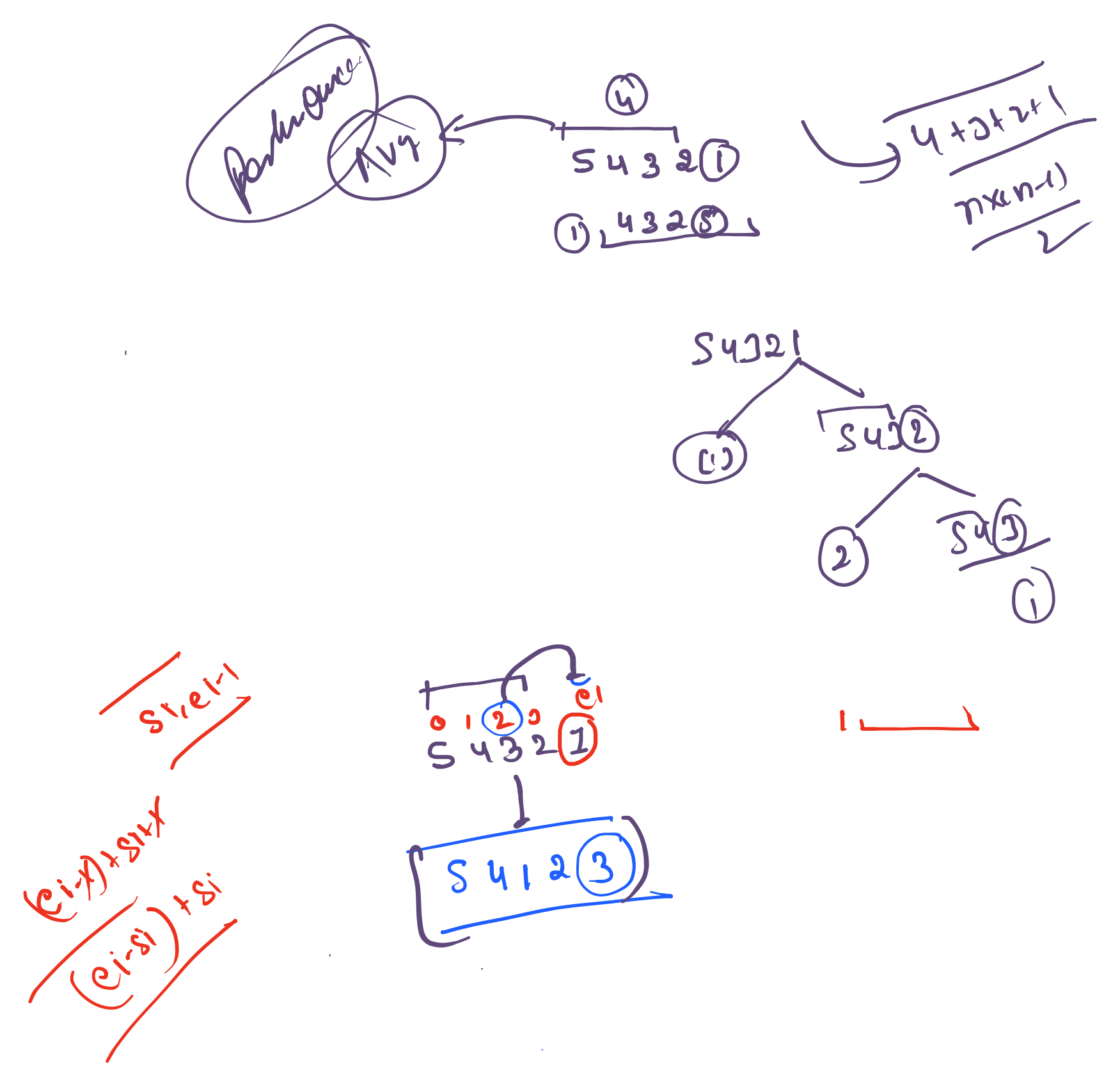

si idx
idx+1 ei
[2 1] 4 7 3 8
idx

```
public static void Sort(int[] arr, int si, int ei) {
    if (si >= ei) {
        return;
    }
    int idx = Partition(arr, si, ei);
    Sort(arr, si, idx - 1);
    Sort(arr, idx + 1, ei);
}
```


idx=0 1
7 5 1 2 4
5 7 2 4
1 2 7 5 4
[2 1] (4) [7 5]

```
public static int Partition(int[] arr, int si, int ei) {
    int item = arr[ei];
    int idx = si;
    for (int i = si; i < ei; i++) {
        if (arr[i] < item) {
            int temp = arr[i];
            arr[i] = arr[idx];
            arr[idx] = temp;
            idx++;
        }
    }
    int temp = arr[ei];
    arr[ei] = arr[idx];
    arr[idx] = temp;
    return idx;
}
```

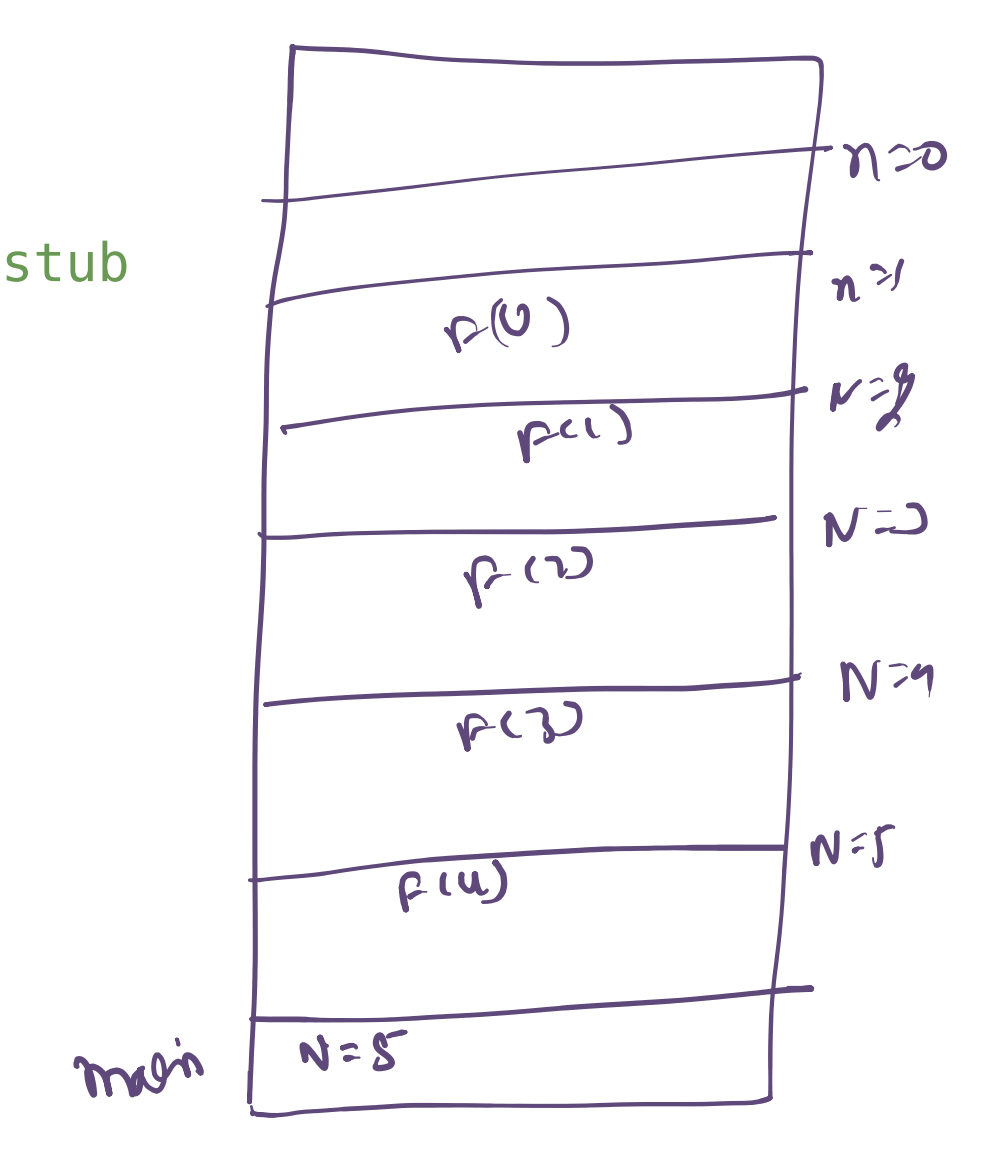

[5 7 2 1 4]
2 7 5 1 4
2 1 5 7 4
2 1 4 7 5
idx


Partition
N-1
4
5 4 3 2 1
1 4 3 2 5
4+3+2+1
n×(n-1)/2
5 4 3 2 1
(1) 5 4 3 2
2 5 4 3
1
si, ei-1
(ei-1)-si+1
(ei-si)+si
0 1 2 3
5 4 3 2 1
5 4 1 2 3

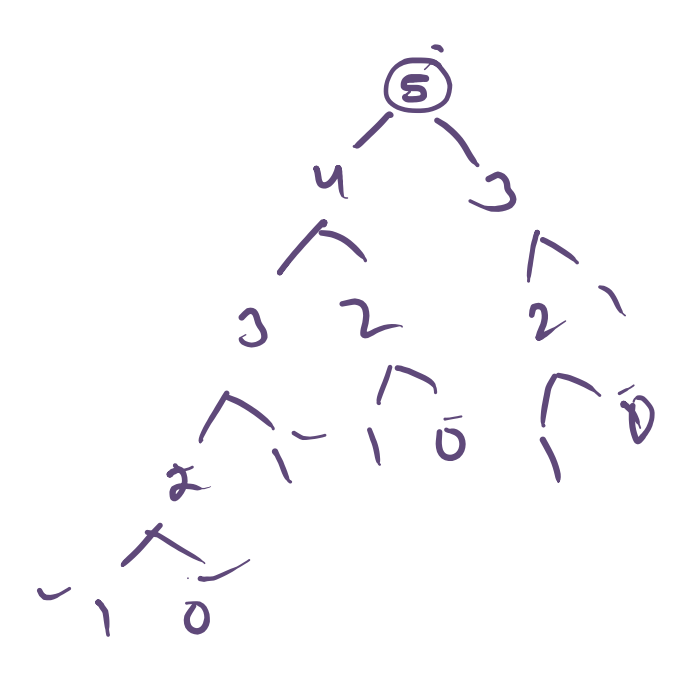
① T.C

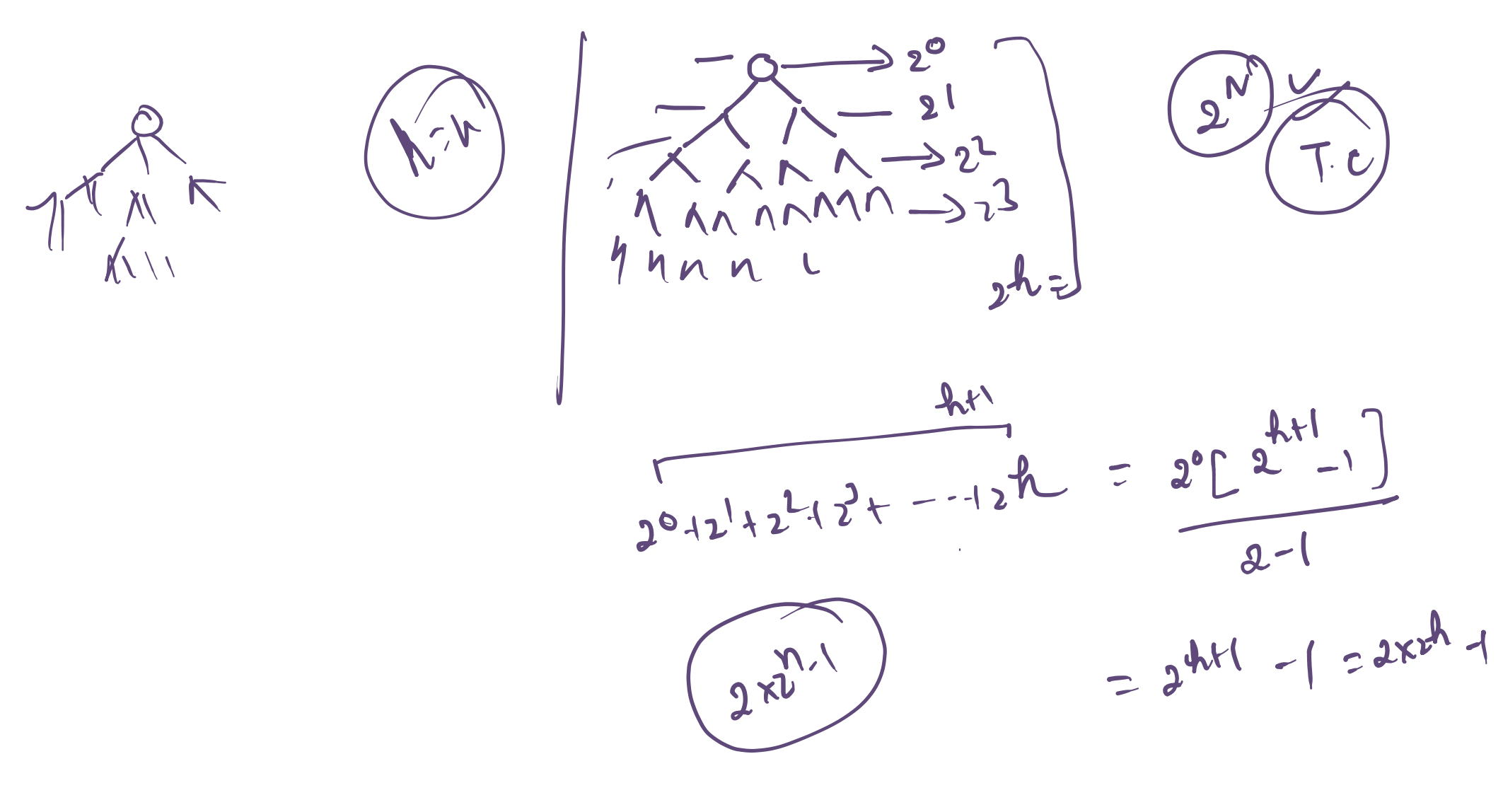
```
public static int fac(int n) {
    // TODO Auto-generated method stub
    if (n == 0) {
        return 1;
    }
    int f = fac(n - 1);// SP
    return f * n;
}
```

O(n)


n=0
n=1
N=2
N=3
N=4
N=5
F(0)
F(1)
F(2)
F(3)
F(4)
main N=5

```
public static int Fib(int n) {
    if (n == 0 || n == 1) {
        return n;
    }
    int f1 = Fib(n - 1);
    int f2 = Fib(n - 2);
    return f1 + f2;
}
```


5
4 3
3 2 2 1
2 1 1 0 1 0
1 0


h:n
2^0
2^1
2^2
2^3
2^h
2^N
T.C

$$2^0+2^1+2^2+2^3+\dots+2^h = \frac{2^0[2^{h+1}-1]}{2-1}$$

$$= 2^{h+1}-1 = 2\times2^h-1$$

$$2\times2^{n-1}$$

[

```
public static int fac(int n) {
    // TODO Auto-generated method stub
    if (n == 0) {
        return 1;
    }
    int f = fac(n - 1);// SP
    return f * n;
}
```

→ T(n) = ?

$T(n) = T(n-1) + 1$

$T(n-1) = T(n-2) + 1$

$T(n-2) = T(n-3) + 1$

$T(n-3) = T(n-4) + 1$

$T(n-4) = T(n-5) + 1$

⋮

$T(1) = 1$

$T(n) = N$

$O(n)$

$T(n) = T(n/2) + 1$

$T(n/2) = T(n/2^2) + 1$

$T(n/2^2) = T(n/2^3) + 1$

$T(n/2^3) = T(n/2^4) + 1$

$T(n/2^4) = T(n/2^5) + 1$

⋮

$T(n/2^k) = 1$

$\frac{N}{2^k} = 1$

$2^k = N$

$k = \log n$

$T(n) = K$

$\le \log(n)$

```
public static int Fib(int n) {
    if (n == 0 || n == 1) {
        return n;
    }
    int f1 = Fib(n - 1);
    int f2 = Fib(n - 2);
    return f1 + f2;
}
```

$T(n) = T(n-1) + T(n-2) + 1$

$T(n-1) = T(n-2) + T(n-3) + 1$

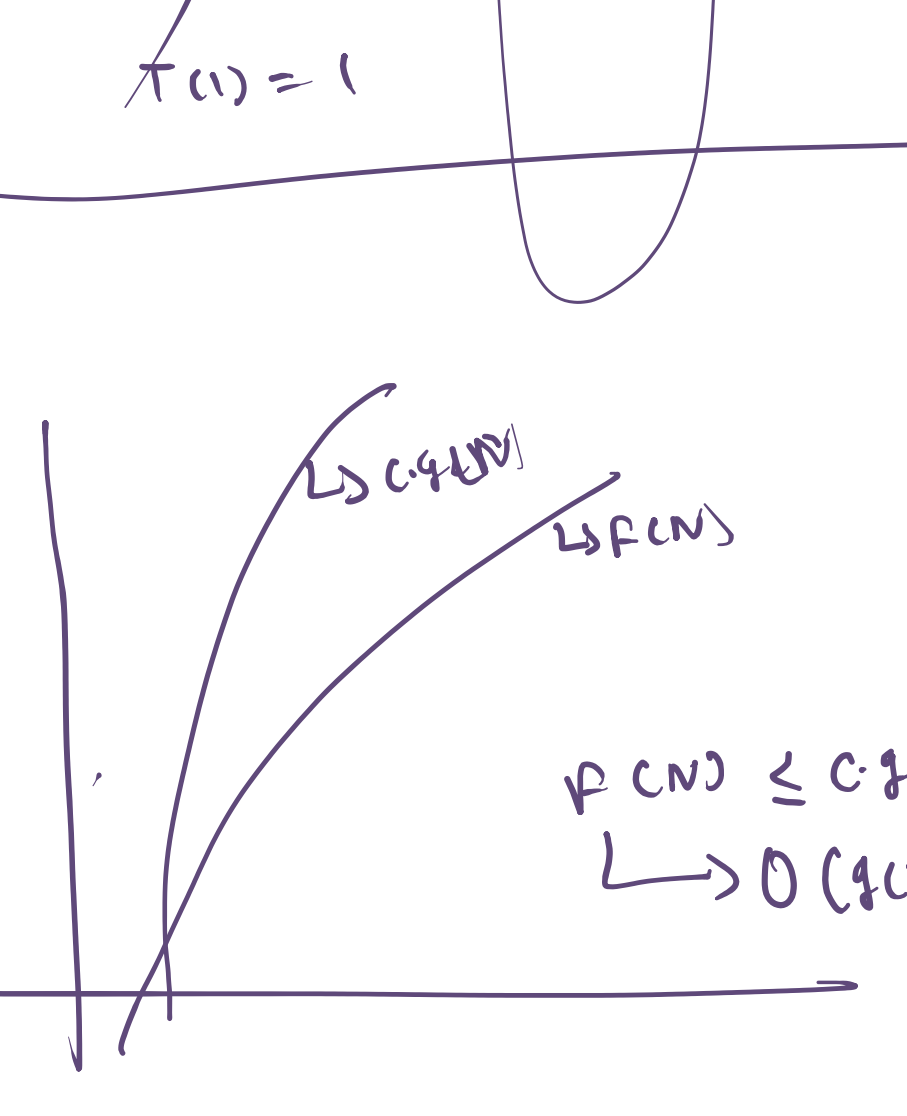
$T(n-2) = T(n-3) + T(n-4) + 1$

$T(n-3) = T(n-4) + T(n-5) + 1$

⋮

$T(1) = 1$

$T(n-1) + T(n-2) \le T(n-1) + T(n-1)$



$f(n) \le c\cdot g(n)$

$\to O(g(n))$

$T(n) = 2T(n-1) + 1$

$2 \quad T(n-1) = 2T(n-2) + 1 \quad \times 2$

$2^2 \quad T(n-2) = 2T(n-3) + 1 \quad \times 2^2$

$2^3 \quad T(n-3) = 2T(n-4) + 1 \quad \times 2^3$

$2^4 \quad T(n-4) = 2T(n-5) + 1 \quad \times 2^4$

$2^5 \quad T(n-5) = 2T(n-6) + 1 \quad \times 2^5$

⋮

$2^{N-1} \quad T(1) = 1 \quad \times 2^{N-1}$

2^{n-1}

$$T(n) = 1 + 2 + 2^2 + 2^3 + \dots + 2^{n-1} = \frac{1[2^N-1]}{2-1} = 2^N-1$$

$T(n) = T(n/2) + T(n/2) + N$

$T(n) = 2T(n/2) + N$

$2 \quad T(n/2) = 2T(n/2^2) + \frac{N}{2} \times 2$

$2^2 \quad T(n/2^2) = 2T(n/2^3) + \frac{N}{2^2} \times 2^2$

$2^3 \quad T(n/2^3) = 2T(n/2^4) + \frac{N}{2^3} \times 2^3$

$2^4 \quad T(n/2^4) = 2T(n/2^5) + \frac{N}{2^4} \times 2^4$

⋮

$2^k \quad T(n/2^k) = \frac{N}{2^k} \times 2^k$

$\frac{N}{2^k} = 1 \to \log(n)$

$T(n) = KN$

$= N\log N$

Pow → log(n)

$P = Pow(a, n/2)$

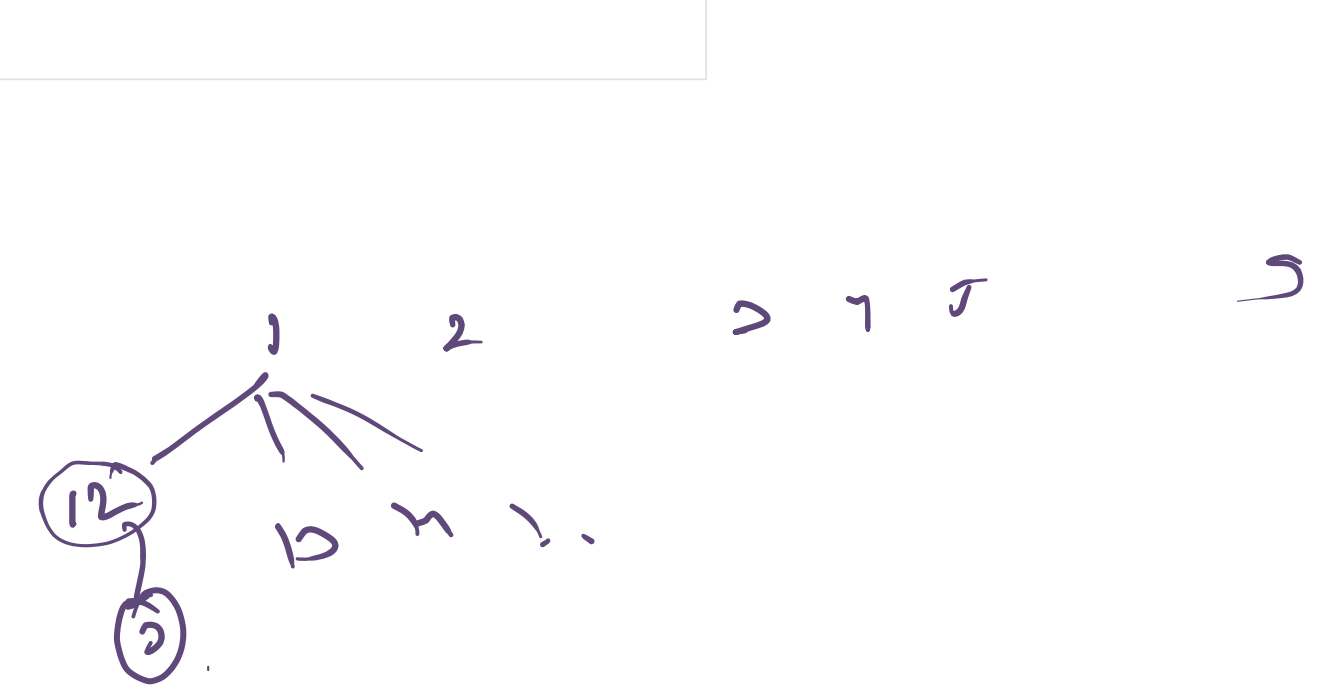
$P \times P =$

$a^9 \to a^4$

```
Pow(int a, int n){
  if(n==0){
   return 1
  P = Pow(a, n/2)
   P = P×P
   if(a%2 != 0){
   P = P×a
)
```

$a^9 \to a^{n/2}$

$a^4 \times a^4 \times a$


1 2 3 4 5
12
3
10 11